PROYECTO 3 · SPRINT 2

INTELLIGENT DOCUMENT ASSISTANT

DOCUMENTO DE ESTUDIO

Cómo se divide el archivo

La analogía del índice: de una página entera a fichas temáticas que conservan su folio y su rango.

FICHA DE RECORTE #002

Manuscrito	**1 × PDF**
Folios	**2 páginas**
Fichas	**6 fragmentos**
Objetivo	**1.200 caracteres**
Solapamiento	**150 caracteres**
Semántica	**Todavía no**
Total	**Sprint 2**

El archivo ya sabe leer.
Ahora aprende a recortar.

Next.js 16 · TypeScript · PostgreSQL · Sin embeddings

Cada ficha recuerda su folio y su rango de caracteres.

LA IDEA CENTRAL

El archivo ya sabe leer; ahora aprende a recortar

En el Sprint 1 el archivo recibió un manuscrito, lo desarmó en folios y guardó cada página con su número. En el Sprint 2 damos el paso siguiente: cada folio se divide en **fichas temáticas** más pequeñas, llamadas **chunks**. La ficha es la unidad que después se va a buscar y citar.

La pregunta del sprint no es «¿cómo buscamos?». Es anterior: **¿cómo cortamos el texto sin perder de dónde salió?** Si un fragmento no conserva su página y su posición, el embedding más sofisticado del mundo no puede citar la fuente. Primero construimos el índice; después llega el bibliotecario.

> **La tesis del Sprint 2:** todavía no necesitamos entender el significado. Necesitamos que cada ficha sea lo bastante chica para representar una idea y lo bastante grande para entenderse sola. Y, sobre todo, que sepa de qué folio salió.

El reparto de roles

EN EL ARCHIVO	EN LA APLICACIÓN	DÓNDE VIVE
Folio completo	El texto de una página, tal como llegó del Sprint 1	`DocumentPage.text`
Ficha temática	Un fragmento con contenido y rango de caracteres	`DocumentChunk`
Regla de corte	El algoritmo puro que decide dónde cortar	`lib/chunking/chunk-text.ts`
Parámetros de la guillotina	Tamaño objetivo, solapamiento y mínimo	`CHUNK_TARGET_CHARS`, `CHUNK_OVERLAP_CHARS`, `CHUNK_MIN_CHARS`
Rehacer el índice	Reemplazar las fichas de un documento sin duplicarlas	`lib/documents/chunks.ts`
Mostrador de fichas	El endpoint que dispara la generación	`POST /api/documents/[id]/chunks`
Sala de consulta	La vista que muestra texto, fichas, página y rango	`DocumentDetail.tsx`, `DocumentChunkViewer.tsx`
Búsqueda semántica	Todavía no existe	Sprint 3 en adelante

LA DECISIÓN DE FONDO

Por qué no alcanza con un vector por documento

Un embedding convierte texto en un vector: una lista de números que representa, a grandes rasgos, su significado. Si representamos un PDF de cientos de páginas con un único vector, ese vector mezcla todos los temas del documento. La búsqueda pierde precisión y la cita se vuelve imposible.

DOCUMENTO ENTERO

Un vector, muchos temas

Un manual de 200 páginas puede hablar de instalación, seguridad, facturación y errores. Un solo vector no puede señalar en cuál de esos temas está la respuesta.

CHUNKS

Un vector por idea

Cada fragmento representa una sección más acotada. Al preguntar, se recuperan solo los fragmentos que parecen pertinentes y se puede citar su origen.

> **La analogía del catálogo:** una ficha por libro alcanza para saber que el libro existe. Para encontrar una respuesta concreta, conviene una ficha por capítulo o por sección. El chunk es esa ficha intermedia: no tan grande como el libro, no tan chica como una palabra.

La división también tiene que conservar contexto. Si cortamos justo entre una pregunta y su respuesta, el fragmento recuperado puede resultar incomprensible por sí solo. Por eso el algoritmo no corta en cualquier posición: baja por la estructura del texto y agrega un pequeño solapamiento entre fichas vecinas.

1.200	150	100	1
Caracteres objetivo por fragmento	Caracteres de solapamiento	Mínimo antes de fusionar el final	Página por fragmento

Los valores son parámetros de partida, no reglas universales. En producción se ajustan al modelo de embeddings y se comparan con preguntas de evaluación.

EL ALGORITMO

Primero párrafos, después oraciones, al final palabras

El chunker no corta en el carácter 1.200 exacto. Busca el límite más significativo que puede pagar: si un párrafo entra, lo conserva; si no, intenta una oración; si tampoco, baja al nivel de palabra. Es una jerarquía de tijeras.

1

Normalizar el texto

En el archivo: alisar el papel antes de recortar. Las arrugas no cambian el contenido, pero hacen que las medidas no coincidan.

En la app: `normalizeText` unifica finales de línea, convierte espacios especiales, colapsa espacios repetidos y deja como máximo una línea en blanco. No destruye el sentido; deja el texto comparable y estable para calcular offsets.

2

Cortar y unir párrafos

En el archivo: separar el manuscrito por párrafos y armar fichas con los párrafos que entren juntos en una misma tarjeta.

En la app: los párrafos se detectan con `\n\n`. Si un párrafo entra en `CHUNK_TARGET_CHARS`, se guarda entero. Después se van agregando párrafos vecinos mientras el fragmento no supere el objetivo.

3

Bajar a oraciones

En el archivo: si un párrafo no entra en la ficha, se corta después de un punto, una interrogación o una exclamación. Nunca en medio de una idea.

En la app: si un párrafo supera el objetivo, `sentenceRanges` lo divide después de `.`, `?`, `!` o `…` seguidos de espacio. Las oraciones se empaquetan como unidades.

4

Bajar a palabras

En el archivo: si una oración tampoco entra, se corta entre palabras. Es la última tijera razonable antes de partir una palabra.

En la app: `wordRanges` recorre las palabras con `\S+`. Si el fragmento todavía no entra, se corta en los espacios. El objetivo se mide sobre el contenido nuevo, no sobre el texto que se repite por overlap.

EL ALGORITMO · CONTINUACIÓN

La red de seguridad y el orden final

5

Una palabra gigante no puede bloquear todo

En el archivo: si una sola palabra ocupa más que la ficha, el archivero la parte en pedazos exactos. Es feo, pero garantiza que el trabajo termine.

En la app: si una palabra supera el objetivo, se corta a la fuerza en trozos de `targetChars`. Eso garantiza **progreso**: el algoritmo nunca se queda mirando la misma palabra para siempre.

6

No dejar una ficha huérfana al final

En el archivo: si la última ficha tiene apenas una línea, se pega a la anterior. Una ficha diminuta no aporta contexto y complica el índice.

En la app: si el último chunk mide menos que `CHUNK_MIN_CHARS`, se fusiona con el anterior. Puede quedar un poco más grande que el objetivo; se acepta para no perder el final de la página.

7

Agregar el solapamiento

En el archivo: la ficha siguiente repite el final de la anterior para que el lector no pierda el hilo si el corte cayó entre una pregunta y su respuesta.

En la app: `applyOverlap` retrocede hasta `CHUNK_OVERLAP_CHARS` y avanza hasta el comienzo de la palabra siguiente. El contenido del chunk es siempre un slice exacto del texto normalizado.

El recorrido dentro del módulo puro

```
chunkText(raw, options)
├── normalizeText(raw)                ← espacios y saltos consistentes
├── buildUnits(text, target)
│   ├── paragraphRanges(text)         ← \n\n
│   ├── sentenceRanges(text, párrafo) ← . ? ! …
│   └── wordRanges(text, oración)     ← \S+ y corte de palabras gigantes
├── packUnits(units, options)         ← unir unidades hasta el objetivo
│   └── fusionar el último si < minChars
└── applyOverlap(text, packed, overlap)← retroceder a un inicio de palabra
```

La garantía del algoritmo: cada paso produce unidades no vacías y cada chunk avanza en el texto. No hay bucles infinitos, no hay chunks de solo espacios y no hay fragmentos que crucen de una página a otra.

Un detalle real del pipeline: PDF.js emite un salto por línea, pero no conserva las líneas en blanco. Por eso el texto extraído suele no tener `\n\n` y el chunker ve un solo párrafo por página, cortando por oraciones. La lógica por párrafos está probada con texto sintético y funciona cuando el texto los conserva. Mejorar la detección de párrafos en la extracción queda para otro sprint.

EL SOLAPAMIENTO

Repetir el final para no perder el hilo

El **overlap** es una pequeña parte del final de un fragmento que se repite al principio del siguiente. No es duplicar filas: es hacer que los rangos se pisen a propósito para conservar el contexto que cruza el límite.

```
Texto normalizado de una página (2.595 caracteres)

Chunk 0  [0 ──────────────────────────── 1109]
Chunk 1             [966 ──────────────────────────── 2260]
Chunk 2                                  [2112 ──── 2595]

overlap 0 → 1: 143 caracteres
overlap 1 → 2: 148 caracteres
límite configurado: 150 caracteres
```

El overlap real puede ser un poco menor que el configurado. Si el punto exacto cae en medio de una palabra, el algoritmo avanza hasta el comienzo de la palabra siguiente. Es preferible perder unos caracteres de solapamiento antes que entregar una palabra partida.

SIN OVERLAP

El corte es una pared

Si una pregunta termina en un fragmento y la respuesta empieza en el siguiente, el primero puede quedar incompleto y el segundo puede carecer de contexto.

CON OVERLAP

El corte es un puente

La ficha siguiente repite el final de la anterior. El fragmento recuperado conserva la idea completa y la cita sigue apuntando a la misma página.

> **El objetivo se mide sobre el contenido nuevo.** Si el objetivo es 1.200 y el overlap es 150, un chunk con solapamiento puede medir hasta 1.350 caracteres. Es una decisión deliberada: el overlap no compite con el objetivo, lo complementa.

LA TRAZABILIDAD

Cada ficha recuerda su folio y su rango

Un chunk no es un string suelto. Tiene una página de origen, un orden dentro de esa página y un rango de caracteres. Esa cadena es la que permitirá responder «¿de dónde salió esto?» mucho antes de que exista un LLM.

EL FOLIO

DocumentPage

- `id`
- `documentId`
- `pageNumber`
- `text` normalizado
- `unique(documentId, pageNumber)`

1 — N

LA FICHA

DocumentChunk

- `id`
- `documentPageId`
- `chunkIndex`
- `content`
- `startChar`, `endChar`
- `createdAt`
- `unique(documentPageId, chunkIndex)`
- `onDelete: Cascade`

El invariante que hace verificable la cita

El contenido de cada chunk es exactamente el rango `[startChar, endChar)` del texto normalizado de su página. No es una reconstrucción aproximada: es un slice.

```
content === normalizedPageText.slice(startChar, endChar)
```

Eso permite mostrar una cita como **«documento.pdf, página 1, caracteres 0–1109»** y, más adelante, resaltar exactamente esa porción del texto. La verificación con SQL es directa:

```
SELECT count(*) AS content_not_equal_to_slice
FROM "DocumentChunk" c
JOIN "DocumentPage" p ON p.id = c."documentPageId"
WHERE c.content <> substring(
  p.text FROM c."startChar" + 1
  FOR c."endChar" - c."startChar"
);
-- Resultado esperado: 0
```

> **La cadena completa:** `Document` → `DocumentPage` → `DocumentChunk`. Si se borra el documento, la cascada borra sus páginas y sus chunks. Si se borra una página, desaparecen solo sus chunks. No quedan fichas huérfanas.

Los parámetros no se guardan por documento. La vista muestra la configuración vigente del servidor. Si cambiás las variables después de generar, los chunks existentes conservan sus rangos anteriores hasta que vuelvas a regenerar. Es una decisión consciente para no agregar campos fuera del alcance del sprint.

LA OPERACIÓN REPETIBLE

Regenerar no debe duplicar fichas

La acción se puede ejecutar muchas veces. El resultado tiene que ser el mismo conjunto de fragmentos, no una pila de copias. Esa propiedad se llama **idempotencia**.

NAVEGADOR

1. «Generar fragmentos»

La isla cliente hace `POST /api/documents/[id]/chunks`. No manda parámetros: el servidor decide el tamaño y el overlap.

↓

FRONTERA CLIENTE / SERVIDOR

SERVIDOR · ROUTE HANDLER

2. Validar el id

Un id con formato inválido devuelve **400**; un id válido que no existe devuelve **404**. El error interno queda en la terminal, nunca en la respuesta.

↓

SERVIDOR · TRANSACCIÓN

3. Borrar las fichas viejas

Dentro de `prisma.$transaction`, se eliminan solo los chunks de las páginas de ese documento. El manuscrito y los folios no se tocan.

↓

SERVIDOR · CHUNKING

4. Generar y guardar

Cada página pasa por `chunkText` con la configuración vigente. Los fragmentos se insertan con `chunkIndex` 0..n-1 y sus offsets.

↓

NAVEGADOR · RESULTADO

5. Reemplazar el estado

La respuesta trae los chunks agrupados por página. La UI actualiza la vista y muestra cuántos se generaron, o el error seguro.

Qué pasa si una inserción falla

La transacción revierte el borrado. Los chunks anteriores vuelven a estar ahí, como si nada hubiera pasado. Lo verifiqué forzando un fallo con un trigger temporal: el endpoint respondió **500** con un mensaje seguro y los 6 chunks anteriores sobrevivieron. Después de quitar el trigger, la regeneración volvió a funcionar.

SITUACIÓN	RESPUESTA	QUÉ VE LA PERSONA
Id con formato inválido	`400 INVALID_DOCUMENT_ID`	«El identificador del documento no es válido.»
Documento inexistente	`404 DOCUMENT_NOT_FOUND`	«Documento no encontrado.»
Método incorrecto	`405`	Next.js rechaza el verbo no soportado.
Fallo inesperado	`500 INTERNAL_ERROR`	«No se pudieron generar los fragmentos.»

> **La regla:** regenerar chunks nunca modifica ni borra documentos o páginas. Solo reemplaza las fichas derivadas. La fuente queda intacta.

LA INTERFAZ

Qué ves en la sala de lectura

La vista de detalle conserva el texto extraído de cada página y agrega una acción para generar o regenerar fragmentos. Los chunks aparecen dentro de su página, en orden, con el rango y los parámetros usados.



ELEMENTO	QUÉ MUESTRA
Botón	«Generar fragmentos» o «Regenerar fragmentos». Se deshabilita si el documento no tiene texto.
Parámetros	Tamaño objetivo, overlap y mínimo. Son la configuración vigente del servidor.
Página	El texto extraído y la cantidad de caracteres. Es el folio del Sprint 1.
Fragmento	Índice, página de origen, rango `startChar–endChar` y contenido.
Página vacía	«Esta página no tiene texto; no se generan fragmentos.»
Sin chunks	«Esta página todavía no tiene fragmentos generados.»
Estados	Cargando, éxito con el conteo, vacío y error seguro.

> **El navegador no decide los parámetros.** La isla cliente solo dispara la acción y muestra el resultado. El servidor lee la configuración, valida y genera. No se confía en el cliente para el tamaño ni para el overlap.

LO QUE TODAVÍA NO HACE

Los límites que conviene revisar

Ninguno de estos límites rompe la trazabilidad. Son decisiones de alcance y puntos para el próximo sprint.

LÍMITE	QUÉ SIGNIFICA	QUÉ HACER
Párrafos de PDF	PDF.js no conserva líneas en blanco; el chunker suele ver un párrafo por página.	Mejorar la detección de párrafos en la extracción en otro sprint.
Caracteres ≠ tokens	1.200 caracteres es una aproximación; cada modelo tokeniza distinto.	Ajustar con el tokenizador del modelo de embeddings y evaluar recuperación.
Parámetros no persistidos	La UI muestra la configuración actual, no la usada en una generación vieja.	Regenerar después de cambiar las variables, o persistir los parámetros más adelante.
Fusión del mínimo	El último chunk puede superar el objetivo por menos de `minChars`.	Es preferible a perder el final de la página.
Overlap	Puede ser menor que el configurado para no cortar palabras; el chunk final puede superar el objetivo.	Documentar y medir el efecto en la recuperación.
Detección de oraciones	Abreviaturas como «Dr.» pueden tratarse como fin de oración.	Aceptable para este sprint; se puede refinar con un tokenizador de oraciones.
Límite de página	Un párrafo que continúa en la página siguiente se corta en ese límite.	Aceptado: la cita siempre apunta a una sola página.

LÍMITE	QUÉ SIGNIFICA	QUÉ HACER
Regeneración concurrente	Dos pedidos simultáneos no están serializados con un lock explícito.	La transacción y el unique protegen la integridad; uno podría fallar con un 500 seguro.

EMBEDDINGS

No hay vectores todavía

No se llama a ningún modelo, no se guarda un vector y no se consulta pgvector. El Sprint 2 solo prepara los fragmentos.

BÚSQUEDA

No hay búsqueda semántica

No hay similitud, búsqueda híbrida ni reranking. La única forma de ver los chunks es abrir el documento.

CHAT

No hay respuestas generativas

No hay LLM, ni prompt con contexto, ni citas generadas. Eso llega después de elegir el modelo y medir la recuperación.

PRODUCCIÓN

No hay auth ni límites de uso

La versión sigue siendo local. No hay autenticación, permisos por documento, rate limiting ni observabilidad.

HOJA DE RUTA

Qué se agrega después del chunking

La arquitectura está preparada para crecer por partes. El Sprint 2 no busca; deja la materia prima lista para que la búsqueda tenga sentido.

SPRINT	SE AGREGA	EN EL ARCHIVO	QUEDA IGUAL
3	Embeddings y pgvector	Un vector por chunk y un índice de similitud	La página, el rango y el contenido del chunk
4	RAG y OpenRouter	Recuperar fragmentos y responder con citas	La trazabilidad y la sala de lectura
5	Evaluación, búsqueda híbrida y reranking	Un control de calidad de la recuperación	El pipeline de ingesta y el contrato
6	Auth, permisos e IDOR	Credenciales y documentos privados	El recorrido de carga, chunking y lectura
7	Producción y optimización	Límites, costos, observabilidad y despliegue	La separación por capas

El próximo paso: elegir un modelo de embeddings, definir la dimensión del vector y medir la recuperación con preguntas de evaluación. El tamaño de chunk se ajustará con el tokenizador del modelo elegido, no con una regla universal.

EL MAPA DE ARCHIVOS

Dónde vive cada parte del recorte

Si mañana tenés que explicar el Sprint 2, este es el orden en el que conviene abrir los archivos.

```
src/
├── app/
│   ├── api/documents/[id]/chunks/route.ts  ← el mostrador de fichas
│   ├── documents/[id]/page.tsx             ← la sala de lectura
│   └── components/documents/
│       ├── DocumentChunkViewer.tsx         ← la isla cliente
│       └── DocumentDetail.tsx              ← la composición
├── lib/
│   ├── chunking/
│   │   ├── chunk-text.ts                   ← el algoritmo puro
│   │   └── config.ts                       ← los parámetros
│   └── documents/
│       ├── chunks.ts                       ← la transacción
│       ├── queries.ts                      ← la lectura con chunks
│       ├── dto.ts                          ← el contrato JSON
│       └── types.ts                        ← los tipos compartidos
└── tests/
    └── chunking.test.mjs                   ← los 12 casos del algoritmo

prisma/schema.prisma                        ← Document, DocumentPage, DocumentChunk
prisma/migrations/20261111203342_add_document_chunk/
```

Checklist del Sprint 2

- [ ] Podés generar fragmentos para un documento y verlos en orden.
- [ ] Cada fragmento conserva su página y su rango de caracteres.
- [ ] Regenerar no crea duplicados.
- [ ] El algoritmo respeta párrafos y oraciones cuando puede.
- [ ] Los textos largos tienen una salida por palabras y por corte forzado.
- [ ] La UI muestra carga, éxito, vacío y error.
- [ ] No hay embeddings, búsqueda semántica ni llamadas a un LLM.

Cómo probarlo

Levantá el servidor, subí un PDF y abrí su detalle. También podés probar el endpoint directamente:

```
npm run dev
npm run test:chunking

# Generar o regenerar:
curl -X POST http://localhost:3000/api/documents/<id>/chunks
```

PARA COMPROBAR QUE ENTENDISTE

Seis preguntas de repaso

Estas son las preguntas que conviene poder responder sin mirar el código. Si salen, entendiste el recorte y la trazabilidad.

1. ¿Qué problema puede aparecer si representamos un PDF entero con un solo embedding?

El vector mezcla todos los temas del documento. La búsqueda pierde precisión, no puede señalar una sección concreta y la cita se vuelve imposible. Un manual de 200 páginas no cabe en una sola idea.

2. ¿Qué aporta el overlap entre chunks?

Conserva el contexto que cruza el límite. Si una pregunta y su respuesta quedan separadas, el fragmento siguiente repite el final del anterior y la idea no se pierde. No duplica filas: son rangos que se pisan a propósito.

3. ¿Por qué conviene intentar cortar por párrafos u oraciones antes que por una cantidad fija de caracteres?

Porque los párrafos y las oraciones son unidades de sentido. Un corte fijo puede partir una idea al medio; un corte estructural produce fragmentos coherentes y más fáciles de recuperar.

4. ¿Qué información mínima necesitamos guardar para mostrar la página de origen de un chunk?

`documentPageId` para la página, `chunkIndex` para el orden, `content` para el texto y `startChar` / `endChar` para el rango. El número de página sale de la relación con `DocumentPage`.

5. ¿Por qué regenerar chunks debería ser idempotente?

Porque el botón se puede pulsar muchas veces. Sin idempotencia, cada ejecución dejaría copias, el conteo crecería y la recuperación devolvería fragmentos repetidos. La transacción borra y recrea; el unique es la segunda red de seguridad.

6. ¿Por qué los 1.200 caracteres son un punto de partida y no una regla universal?

Porque un carácter no es un token: cada modelo tokeniza distinto según el idioma, los acentos y los signos. El tamaño óptimo depende del modelo de embeddings, de sus límites y de la calidad de recuperación medida con preguntas reales.

> **La respuesta corta:** el chunking no busca todavía. Prepara fichas pequeñas, trazables y repetibles para que la búsqueda del Sprint 3 tenga algo digno de indexar.

Para revisar el código

1. `src/lib/chunking/chunk-text.ts` — el algoritmo puro.
2. `tests/chunking.test.mjs` — los casos que fijan el comportamiento.
3. `src/lib/chunking/config.ts` — parámetros y validación.
4. `src/lib/documents/chunks.ts` — transacción e idempotencia.
5. `src/app/api/documents/[id]/chunks/route.ts` — validación y errores.
6. `prisma/schema.prisma` — relaciones y restricciones.
7. `src/components/documents/DocumentChunkViewer.tsx` — estados de la isla cliente.

GLOSARIO

Los términos que conviene tener a mano

Chunk

Un fragmento de texto más pequeño que una página. Es la unidad que después se vectoriza y se recupera. En este sprint conserva su página y su rango.

Target

El tamaño objetivo de contenido nuevo de cada chunk, medido en caracteres. No incluye el overlap.

Overlap

La parte del final de un chunk que se repite al principio del siguiente. Sirve para conservar contexto en el límite.

Offset

La posición de un carácter dentro de un texto. `startChar` y `endChar` son offsets dentro del texto normalizado de la página.

Token

Una unidad de texto que usa un modelo para tokenizar. No equivale a un carácter: depende del modelo y del idioma.

Idempotencia

La propiedad de una operación que, ejecutada varias veces, produce el mismo estado final. Regenerar chunks es idempotente.

Transacción

Un grupo de operaciones que se confirman o se revierten juntas. Evita dejar fichas nuevas a medio guardar.

Cascada

La eliminación automática de los registros hijos cuando se borra el padre. Borrar una página elimina sus chunks.

Embedding

Una representación numérica del significado de un texto. Permite buscar por similitud semántica. Llega en el Sprint 3.

pgvector

Una extensión de PostgreSQL para guardar vectores y buscar por distancia. La imagen de Docker ya es compatible, pero este sprint no la usa.

RAG

Retrieval-Augmented Generation: recuperar fragmentos relevantes y usarlos como contexto para generar una respuesta. Es el objetivo del Sprint 4.

DocumentChunk

La entidad que guarda cada fragmento: página, índice, contenido, offsets y fecha de creación. Es la ficha temática del archivo.

> **La frase para llevarte:** el Sprint 1 construyó el archivo. El Sprint 2 construyó el índice de fichas. Todavía no hay bibliotecario, pero cada ficha ya sabe de qué folio salió y qué parte del texto representa. Cuando llegue la búsqueda semántica, no va a tener que adivinar el origen.